

Crop Stress Detection & Field Advisory Platform

Product Requirements Document (PRD) — Build from the Existing 3D Agricultural Foundation

Project status: The 3D geospatial foundation is already built. This PRD defines the next development needed to turn it into the complete smart-agriculture intelligence system.

Current foundation: Real 500m × 500m agricultural area in Maharashtra, Sentinel-2 imagery, NDVI, DEM/elevation, GIS alignment, CesiumJS 3D visualization, and a planned 10m analysis grid.

1. Product Vision

Build a field-level agricultural intelligence platform that continuously combines satellite imagery, NDVI, terrain, weather and later real/simulated soil observations to detect crop stress, explain likely causes, predict emerging risks, identify affected zones, and provide practical farmer recommendations.

2. What We Already Have

- 500m × 500m real agricultural area in Maharashtra
- Sentinel-2 satellite imagery
- NDVI visualization
- DEM/elevation and 3D depth
- GIS/geographic alignment
- CesiumJS interactive 3D map
- 10m × 10m grid planned (2,500 cells)

Important: The 3D map is the foundation, not the final product.

3. Final User Experience

A farmer or reviewer should be able to open the farm, see the 3D field, switch analytical layers, select a 10m cell/zone, view its current and historical indicators, see risk levels and reasons, and receive an actionable advisory. The platform should support continuous re-analysis as new data arrives.

4. Core Workflow

Register/define field → collect satellite + NDVI + DEM + Open-Meteo data → create 10m grid → calculate cell features → compare historical trends → detect anomalies/stress → estimate severity/risk → visualize risk on 3D field → generate explanation → generate recommendation → alert/dashboard → monitor again.

5. Data Sources

Sentinel-2: Use the existing Sentinel Hub/Copernicus credentials. Sentinel Hub provides APIs for satellite imagery, catalog search and statistical analysis. ■cite■turn0search1■turn0search6■

Weather — Open-Meteo only: No separate API key is required for the standard non-commercial API. The API supports temperature, humidity, precipitation, soil moisture, soil temperature, ET■ and other variables. ■cite■turn0search0■

DEM: Continue using the existing DEM pipeline. Derive elevation, slope and aspect for analysis.

Soil: Phase 1 can use simulated/placeholder sensor readings; Phase 2 can connect ESP32/field sensors for real soil moisture and other measurements. Do not pretend modelled weather soil moisture is equivalent to a physical sensor.

Crop/field metadata: crop type, planting date, crop age/stage, field boundary and optional irrigation information.

6. Required 10m Grid

Create a 10m × 10m grid clipped to the farm boundary. Each cell should have a stable cell_id and geometry/centroid. For a 500m × 500m square, the theoretical grid contains 2,500 cells before clipping to the actual field boundary.

7. Cell-Level Data Model

Each cell should eventually store: cell_id, latitude, longitude, geometry, timestamp, NDVI, previous NDVI values, NDVI change/trend, elevation, slope, aspect, temperature, humidity, precipitation, wind, ET, weather forecast indicators, soil moisture when available, crop type/stage, stress score, stress type, severity, confidence, recommendation, and model version.

8. Historical Intelligence

Do not rely on a single satellite image. Build a time series. Store every processed observation with acquisition time. Calculate 7/14/30-day trends where data availability permits. Detect persistent decline, sudden change and abnormal cells relative to the field baseline.

9. Crop Stress Engine

Initial categories: **water stress, possible disease risk, nutrient-stress indication, pest-risk indication, and general vegetation anomaly**. Phase 1 should use transparent rules/baselines plus trend analysis. ML should be introduced after a reliable feature dataset exists. The system must label uncertain outputs as risk/indication, not as a confirmed medical/agricultural diagnosis.

10. Prediction Layer

The long-term goal is early warning: estimate whether a cell/zone is likely to enter stress based on NDVI trend, weather forecast, ET, precipitation, temperature, terrain and soil observations when available. Start with interpretable risk scoring; later evaluate Random Forest/XGBoost and time-series models against a held-out validation period.

11. Explainable AI

Every alert should answer three questions: **What happened? Why does the system think it happened? What should the farmer check/do?** Example: "Zone 3 shows declining NDVI; recent low precipitation and high ET increase water-stress risk; inspect irrigation in Zone 3." Avoid overconfident treatment instructions.

12. 3D/GIS Intelligence UI

CesiumJS remains the main visualization layer. Add: 10m grid toggle; NDVI layer; NDVI trend layer; elevation/slope layer; weather indicators; stress heatmap; risk severity; cell click/hover panel; timeline/date selector; legend; layer controls; farm summary; alert panel. CesiumJS supports imagery layers and terrain visualization suitable for this role.

■cite■turn0search7■turn0search14■

13. Dashboard Requirements

Farm overview: crop, area, current health score, active alerts.

Map: 3D farm with analytical overlays.

Cell details: current values + historical chart + risk + confidence + explanation.

Trends: NDVI and weather history.

Alerts: urgent/attention/information levels.

Recommendations: practical inspection/irrigation/monitoring actions.

14. Recommended Tech Stack

Frontend: React + Vite + TypeScript + CesiumJS + Tailwind CSS + Recharts/ECharts.

Backend: Python + FastAPI + Pydantic + Uvicorn.

Geospatial: GeoPandas, Shapely, Rasterio, pyproj; PostGIS for production spatial storage.

Data/analysis: NumPy, Pandas, xarray where useful.

ML: scikit-learn first; XGBoost optional after baseline; PyTorch only if a deep-learning model is justified.

Satellite: Sentinel Hub REST APIs and/or sentinelhub-py. Keep the existing working Sentinel pipeline rather than rewriting it unnecessarily. Sentinel Hub's official guide documents Python access and Statistical/Processing APIs.

■cite■turn0search6■

Weather: Open-Meteo REST API only.

Database: PostgreSQL + PostGIS.

Cache/queues: Redis + Celery/RQ only when scheduled processing becomes necessary.

Deployment: Docker + Nginx; frontend on Vercel if appropriate, backend/database on a service supporting containers/PostGIS.

15. Python Libraries

Minimum: **fastapi**, **uvicorn**, **pydantic**, **requests/httpx**, **python-dotenv**, **pandas**, **numpy**, **geopandas**, **shapely**, **rasterio**, **pyproj**, **scikit-learn**, **psycopg2-binary** or **asyncpg**, **sqlalchemy**, **geoalchemy2**.

Satellite integration: **sentinelhub** if compatible with the existing pipeline. Optional: **xarray**, **rioxarray**, **scipy**, **matplotlib** for development/validation.

16. Frontend Libraries

Core: **react**, **typescript**, **vite**, **cesium**, **resium** (if the team prefers React wrappers), **tailwindcss**. Data fetching:

TanStack Query. Forms/validation: **React Hook Form + Zod**. Charts: **Recharts** or **ECharts**. Maps remain Cesium-first; avoid adding another map engine unless a clear requirement appears.

17. Backend Services

Recommended modules:

- **satellite_service.py** — Sentinel search/download/statistics
- **ndvi_service.py** — NDVI calculation/trends
- **weather_service.py** — Open-Meteo integration
- **terrain_service.py** — DEM-derived features
- **grid_service.py** — 10m grid creation/queries
- **feature_service.py** — cell-level feature generation
- **stress_service.py** — risk rules/model inference
- **prediction_service.py** — future-risk estimates
- **recommendation_service.py** — action generation
- **alert_service.py** — alert creation
- **db_service.py** — PostgreSQL/PostGIS access
- **scheduler.py** — periodic data refresh later.

18. Database Design

Core tables: **farms**, **farm_cells**, **satellite_observations**, **weather_observations**, **terrain_features**, **soil_observations**, **crop_profiles**, **cell_features**, **stress_predictions**, **alerts**, **recommendations**, **model_runs**.

Use PostGIS geometry for farm/cell boundaries and spatial queries. Index cell geometry, **farm_id** and observation timestamps.

19. Open-Meteo Integration

Use one weather service: Open-Meteo. The forecast API accepts latitude/longitude and can return hourly/daily variables.

Useful variables for this project include **temperature_2m**, **relative_humidity_2m**, **precipitation**, **wind_speed_10m**, **soil moisture variables**, **evapotranspiration** and **et0_fao_evapotranspiration**. ■cite■turn0search0■

For the 500m farm, begin with the farm centroid for weather. Do not create fake high-resolution weather by assigning independent weather observations to every 10m cell. Use terrain/local sensor data later for finer differentiation.

20. Sentinel Authentication & Secret Management

Keep the existing Sentinel client ID/client secret on the backend only. Do not put secrets in React/Vite client code or GitHub. Sentinel Hub uses OAuth2; access tokens expire and should be refreshed rather than requested for every satellite call. ■cite■turn0search0■turn0search2■

Use a backend `.env` file locally and platform secret/environment-variable storage in deployment. Rotate exposed secrets immediately if they are ever committed to a repository.

21. API Endpoints

Suggested endpoints:

GET /api/farms/{id}
GET /api/farms/{id}/cells
GET /api/cells/{id}/latest
GET /api/cells/{id}/history
GET /api/farms/{id}/ndvi
GET /api/farms/{id}/weather
GET /api/farms/{id}/risk-map
GET /api/farms/{id}/alerts
GET /api/cells/{id}/recommendation
POST /api/analysis/run.

22. Development Phases

Phase 1 — Data foundation: 10m grid, cell IDs, database, Sentinel historical observations, Open-Meteo integration, DEM-derived features.

Phase 2 — Intelligence: NDVI trends, anomaly detection, baseline health/risk score, explainable rules.

Phase 3 — Visualization: interactive cell grid, risk layers, cell details, time series, dashboard.

Phase 4 — Prediction: early-warning model and validation pipeline.

Phase 5 — Advisory: recommendations, alerts, multilingual/voice interface if time permits.

Phase 6 — IoT: ESP32 soil-moisture data and optional automated irrigation integration.

23. ML Strategy

Do not start with a complicated neural network. First create a strong baseline using domain rules + statistical anomaly detection. Then create a labelled/weakly labelled dataset from historical observations and field/event records. Evaluate models using time-aware train/validation splits to avoid leakage. Report precision, recall, F1, confusion matrix and calibration where applicable.

24. Important Limitations

- Satellite observations can be affected by clouds and revisit timing.
- Open-Meteo weather is model-based, not a physical sensor in the field.
- Modelled soil moisture is not equivalent to measured soil moisture.
- NDVI decline does not automatically mean disease; it can result from water, weather, phenology, nutrient issues or other causes.
- Recommendations should be framed as decision support and inspection guidance unless validated by agronomic experts.
- Do not claim yield prediction or disease diagnosis accuracy without validation.

25. Security & Reliability

Never expose Sentinel secrets in frontend code. Validate all farm/cell IDs and coordinates. Add request timeouts/retries for external APIs. Cache weather/satellite results to avoid unnecessary calls. Log model versions and timestamps. Add graceful fallbacks when satellite/weather data are unavailable. Keep external API failures from crashing the Cesium application.

26. Definition of Done

The system is complete for the MVP when a user can: **(1)** open the real Maharashtra farm in 3D, **(2)** see the 10m analytical grid, **(3)** view Sentinel/NDVI history, **(4)** view Open-Meteo weather, **(5)** inspect elevation/slope, **(6)** select a cell and see its features, **(7)** receive a transparent stress/risk assessment, **(8)** see the affected zones in 3D, **(9)** receive an actionable recommendation, and **(10)** rerun/refresh the analysis as new observations arrive.

27. Target Architecture

Browser: React + CesiumJS

↓

API: FastAPI

↓

Spatial DB: PostgreSQL + PostGIS

↓

Processing: Python geospatial + ML services

↓

External data: Sentinel Hub/Copernicus + Open-Meteo

↓

Storage: PostgreSQL/PostGIS + object storage for larger raster artifacts if needed

↓

Scheduled jobs: refresh satellite/weather → compute features → run risk → update alerts.

28. First 10 Tasks to Implement

1. Freeze the current working 3D version in Git.
2. Add PostGIS database.
3. Generate and persist the 10m grid.
4. Attach current NDVI to each cell.
5. Build Sentinel historical retrieval for the same AOI.
6. Build NDVI trend calculation.
7. Add Open-Meteo service and store hourly/daily observations.
8. Extract elevation/slope/aspect per cell.
9. Build first transparent water-stress risk score.
10. Render risk as a selectable 3D layer and show cell-level explanations.

29. Final Product Statement

“An AI-powered field intelligence platform that turns satellite, vegetation, terrain and weather data into cell-level crop monitoring, early risk detection, explainable analysis and actionable farmer recommendations.”

The 3D map is the foundation. The real product is the intelligence built on top of it.